

TABLE OF CONTENTS

Contents

1. The Key Insight: Threads vs Work

2

2. Request Flow -- Step by Step

2

3. Callback Chain Diagram

2

4. Code Walkthrough: thenApply / exceptionally Chain

3

5. Why This Matters Under Load

3

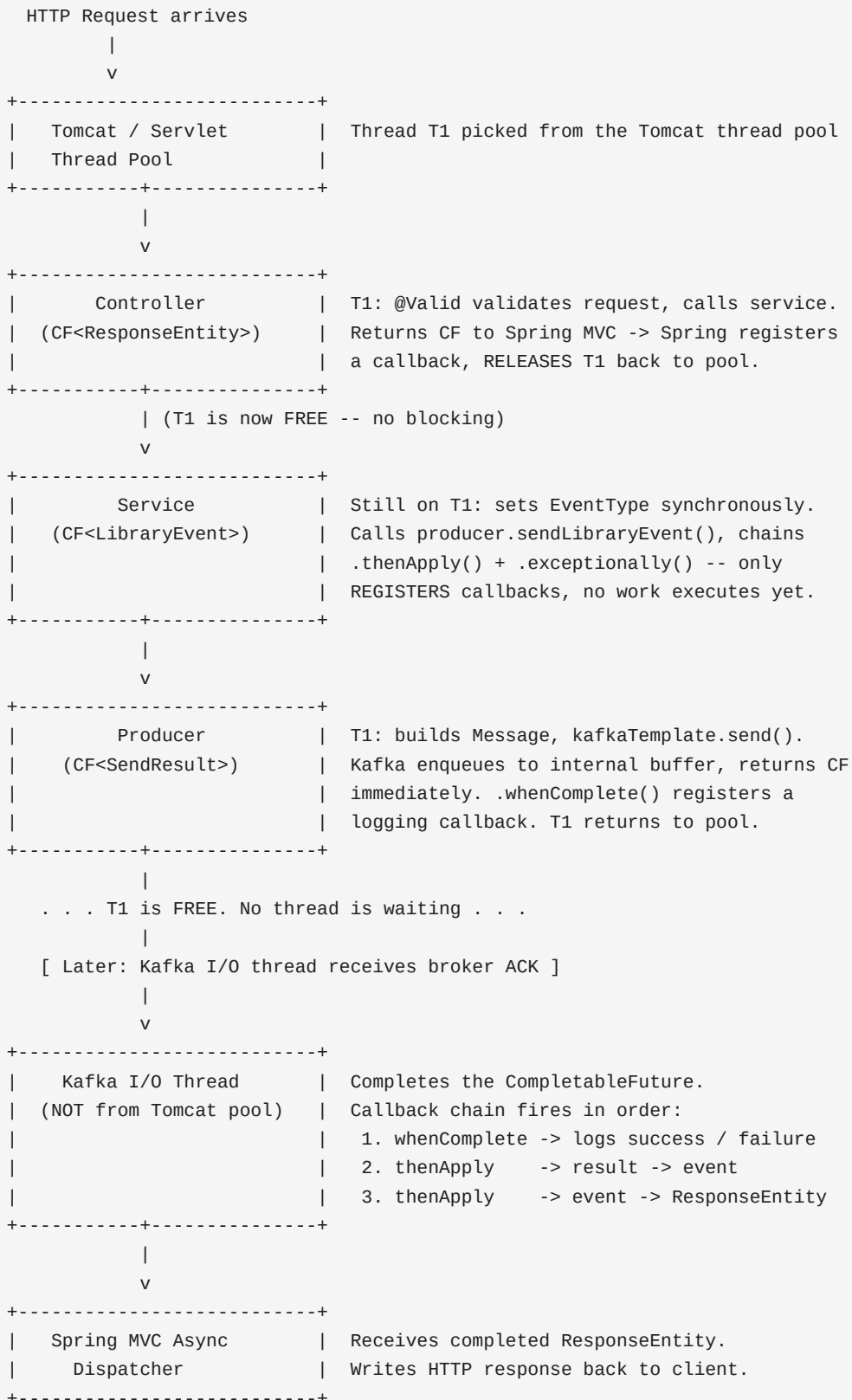
6. Fail Fast -- Event Type Validation Flow

5

1. The Key Insight: Threads vs Work

In a traditional (blocking) approach, one thread does all the work AND waits for the broker ACK before returning. In this implementation, threads hand off work and return to the pool immediately -- the actual I/O wait happens on a Kafka-managed I/O thread, not on a Tomcat Servlet thread.

2. Request Flow -- Step by Step



3. Callback Chain Diagram



4. Code Walkthrough: thenApply / exceptionally Chain

```
return libraryEventProducer.sendLibraryEvent(event) // (1)
    .thenApply(result -> event)                      // (2)
    .exceptionally(ex -> {                          // (3)
        throw new LibraryEventPublishException(
            "Unable to publish ADD library event",
            ex.getCause()                          // (4)
        );
    });
```

(1) `sendLibraryEvent(event)` -- returns `CF<SendResult>` immediately. No waiting. The future completes later when the Kafka I/O thread gets the broker ACK.

(2) `.thenApply(result -> event)` -- when the future completes successfully, ignore the `SendResult` and return the original event. Type changes: `CF<SendResult>` -> `CF<LibraryEvent>`.

(3) `.exceptionally(ex -> { throw ... })` -- if the future completes exceptionally, re-wrap as `LibraryEventPublishException` so `@ControllerAdvice` handles it.

(4) `ex.getCause()` -- `ex` inside `exceptionally` is always a `CompletionException` (Java's internal wrapper). `getCause()` extracts the real root cause.

5. Why This Matters Under Load

Scenario	Blocking (old)	Async (new)
Threads per request	1 held until broker ACK	~0 (microseconds)
100 req @ 50ms Kafka latency	100 threads waiting	Near 0 threads waiting
Throughput ceiling	Thread pool (~200)	Kafka I/O throughput
Tomcat thread usage	High -- proportional to Kafka latency	Minimal -- request parsing only

library-events-producer-v2 has 200 threads. In the blocking approach each request held a thread for the full Kafka round-trip (5-50ms). In the async approach threads are returned to the pool in microseconds, letting the same 200 threads handle thousands of concurrent in-flight Kafka requests.

6. Fail Fast -- Event Type Validation Flow

Even though Bean Validation (@Valid) ensures the eventType is a valid enum value (ADD or UPDATE), it cannot enforce which value is allowed per endpoint. A second guard in the controller provides this check synchronously -- before any async work, service call, or Kafka interaction begins.

```
HTTP POST arrives (e.g. eventType: UPDATE sent to POST endpoint)
|
v
Spring MVC deserialises JSON -> LibraryEvent object
|
v
@Valid runs Bean Validation
- libraryEventId : not null, positive OK
- eventType      : not null, valid enum OK (UPDATE is a valid enum value!)
- book           : valid OK
|
| Bean Validation PASSES -- UPDATE is a valid enum, just wrong endpoint
v
Controller.createLibraryEvent() called on Tomcat thread T1
|
v
if (event.getEventType() != EventType.ADD)
|
v
throw new IllegalArgumentException(...) <-- T1 throws synchronously
|
| Service is NEVER called
| Kafka is NEVER touched
v
Spring MVC catches the exception
|
v
Routes to @ControllerAdvice
handleIllegalArgumentException(ex)
|
v
log.warn("Invalid request argument: ...")
|
v
return ResponseEntity
    .status(400)
    .body(new ErrorResponse(400, ["POST endpoint only accepts ADD events..."]))
|
v
HTTP 400 Bad Request sent to client
```

Why the guard fires before any CompletableFuture is created:

```
// Guard runs synchronously on T1 -- before returning any CF
if (event.getEventType() != EventType.ADD) {
    throw new IllegalArgumentException(...); // T1 throws here
}
// Only reached if eventType == ADD
return libraryEventService.createLibraryEvent(event) // CF created here
    .thenApply(publishedEvent -> ResponseEntity.created(...).body(publishedEvent));
```

Because the exception is thrown synchronously (before a CompletableFuture is ever returned), Spring MVC handles it through its normal synchronous @ExceptionHandler path -- not through async dispatch. @ControllerAdvice catches it

	Without the guard	With the guard (current)
Bean Validation	Passes (UPDATE is valid enum)	Passes
Service called?	Yes -- silently overrides eventType	No
Kafka touched?	Yes -- message published	No
Client informed?	No -- gets 201 with wrong intent	Yes -- 400 immediately
Error path	None (silent data corruption)	@ControllerAdvice -> 400

The guard fires on T1, synchronously, at the earliest possible point -- no CompletableFuture is created, no Kafka buffer is touched, no callback is registered.